Lexical Analysis Specification

Target Language: Subset of JavaScript
Student Name: Waithaka Joseph
Group Number: 19

Gachugu Dylan Gathia SCS3/146838/2023
Ndegwa Jackline Wanjiru SCS3/146683/2023
Waithaka Joseph SCS3/146799/2023
Patrick Njoroge SCS3/146581/2023

1 Lexical Specifications & Regular Expressions

Keywords Reserved structural tokens that dictate control flow and variable bindings within the grammar. Restricted exclusively to three specific strings: `let`, `if`, and `while`.

RE: (`let` | `if` | `while`)

Identifiers Programmer-defined variable names. An identifier must begin with an alphabetic character (upper- or lowercase) or an underscore. Subsequent characters may include any combination of letters, digits, or underscores.

RE: `[a-zA-Z_][a-zA-Z0-9_]*`

Numbers Numeric integer literals used for arithmetic and relational comparisons. Restricted to sequences of one or more decimal digits (0–9).

RE: `[0-9]+`

Operators & Punctuation Single-character syntactic symbols used for assignment, arithmetic, relational comparisons, scoping blocks, and statement termination.

RE: `[=+<(){};]`

2 Nondeterministic Finite Automaton (NFA)

To construct the combined NFA we apply **Thompson's Construction** to each token's regular expression and merge the resulting sub-NFAs under a new start state q_0 via ε-transitions — one ε-branch per token class. This guarantees that every sub-NFA satisfies the structural invariants required by Thompson's Construction:

- Exactly one accepting state per sub-NFA.
- No transitions *out* of any accepting state.

- No transitions *into* any start state.

The states and their roles are as follows:

States	**Token**	**Role**
q_0	—	Combined start (ε-hub)
$q_1 \to q_2 \to q_3 \to q_4$	`let`	Character path: $l \to e \to t$; q_4 accepts
$q_5 \to q_6 \to q_7$	`if`	Character path: $i \to f$; q_7 accepts
$q_8 \to q_9 \to q_{10} \to q_{11} \to q_{12} \to q_{13}$	`while`	Character path: $w \to h \to i \to l \to e$; q_{13} accepts
$q_{14},\ q_{15}$	Identifier	$q_{14} \xrightarrow{\texttt{[a-zA-Z_]}} q_{15}$; q_{15} self-loops on `[a-zA-Z0-9_]`
$q_{16},\ q_{17}$	Number	$q_{16} \xrightarrow{\texttt{[0-9]}} q_{17}$; q_{17} self-loops on `[0-9]`
$q_{18},\ q_{19}$	Op/Punct	$q_{18} \xrightarrow{\texttt{[=+<(){};]}} q_{19}$; q_{19} accepts
q_{err}	Error	Catch-all sink; self-loops on Σ

Because keywords (`let`, `if`, `while`) are also valid prefixes of identifiers, the accepting states of each keyword sub-NFA (q_4, q_7, q_{13}) carry ε-transitions into the shared identifier continuation state q_{15}. This allows the scanner to correctly classify e.g. `letter` as an IDENTIFIER rather than a KEYWORD.

The ε-transition from q_0 to q_{err}, combined with the Σ-self-loop on q_{err}, is the **catch-all rule**: any character not consumed by any other sub-NFA is absorbed by the error sink, preventing the scanner from crashing on unrecognised input.

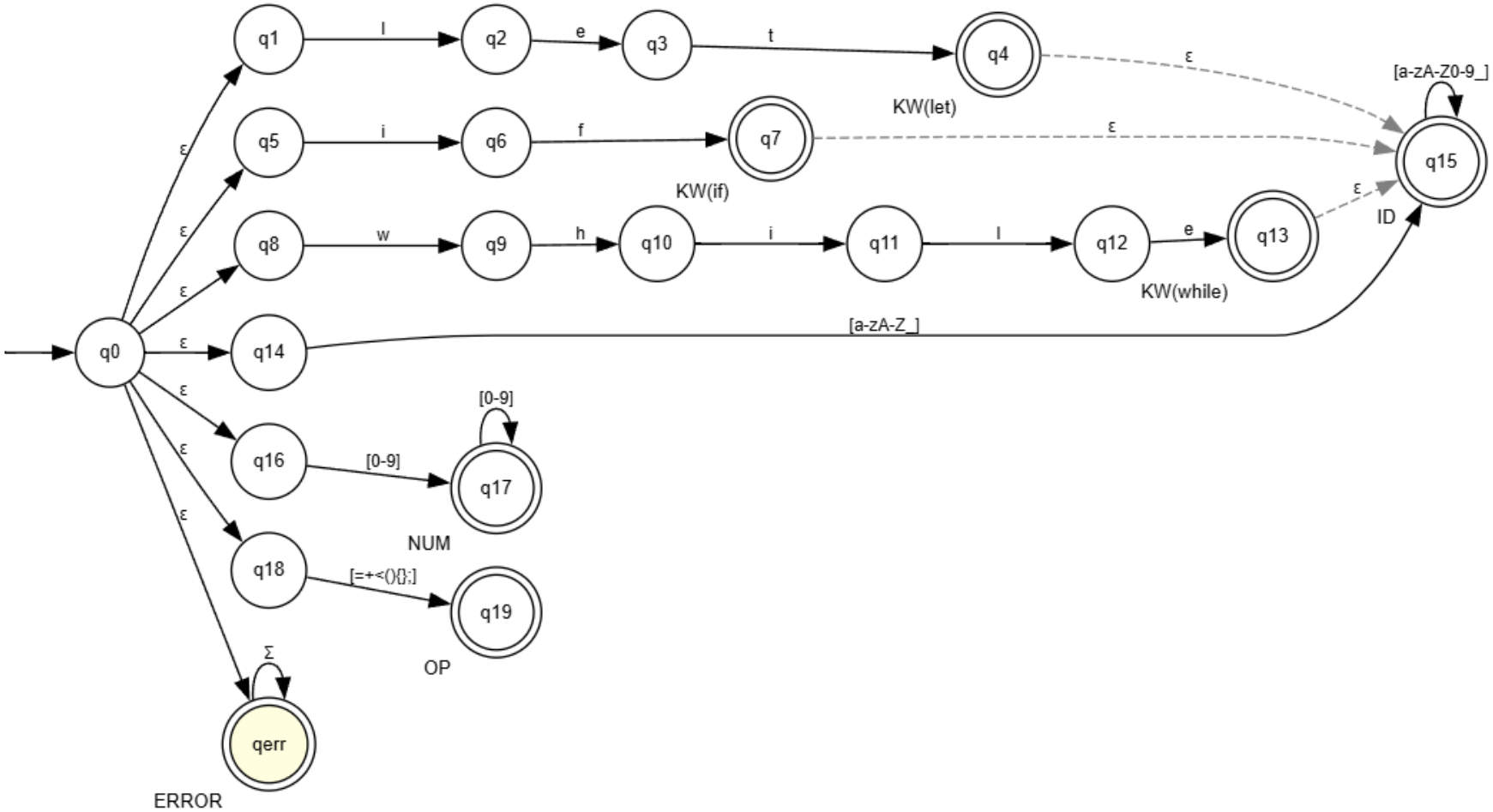

Figure 1: Combined NFA built via Thompson's Construction. ε-transitions fan out from q_0 to each token sub-NFA. All labels use explicit character-class notation: `[a-zA-Z_]`, `[a-zA-Z0-9_]`, `[0-9]`, `[=+<(){};]`. Dashed grey arrows are the ε-links from keyword accepting states into q_{15}, enabling keyword-vs-identifier disambiguation. The yellow q_{err} node is the catch-all error sink.

3 Subset Construction (NFA to DFA Conversion)

We apply the **Subset Construction** algorithm: each DFA state represents the ε-closure of the set of NFA states the machine could simultaneously occupy.

$$\varepsilon\text{-closure}(q_0) = \{q_0,\ q_1,\ q_5,\ q_8,\ q_{14},\ q_{16},\ q_{18},\ q_{\text{err}}\} \longrightarrow \textbf{State A} \text{ (Start)}$$

Reading each possible input symbol from State A, and computing the ε-closure of the resulting NFA state sets, yields all DFA states A–N plus the ERR sink.

Conflict Resolution & Maximal Munch

In our language a string like `let` matches both the *Keyword* regex and the *Identifier* regex. We resolve this using two complementary rules:

1. Maximal Munch. The scanner always consumes the *longest* possible prefix of the remaining input before emitting a token. Ties in length are broken by **rule priority**: keywords are listed before identifiers in the specification, so they win when the match length is equal.

2. Modified Subset Construction. Each DFA accepting state is annotated with the highest-priority token type whose NFA accepting state it contains. States containing both a keyword accepting state *and* q_{15} are labelled KEYWORD.

Concrete example. DFA State K contains NFA states $\{q_4, q_{15}\}$. q_4 is the `let`-keyword accepting state; q_{15} is the identifier accepting state. Because the keyword has higher priority, State K is labelled **Acc: KW(let)**.

- If the scanner has consumed `let` and the next character does not match `[a-zA-Z0-9_]`, the scanner emits `let` as a KEYWORD token.
- If the next character matches `[a-zA-Z0-9_]` (e.g. the input is `letter`), the DFA moves to State E ($\{q_{15}\}$) and keeps consuming. When it eventually sticks, the whole lexeme `letter` is emitted as an IDENTIFIER.

4 Deterministic Finite Automaton (DFA) & State Table

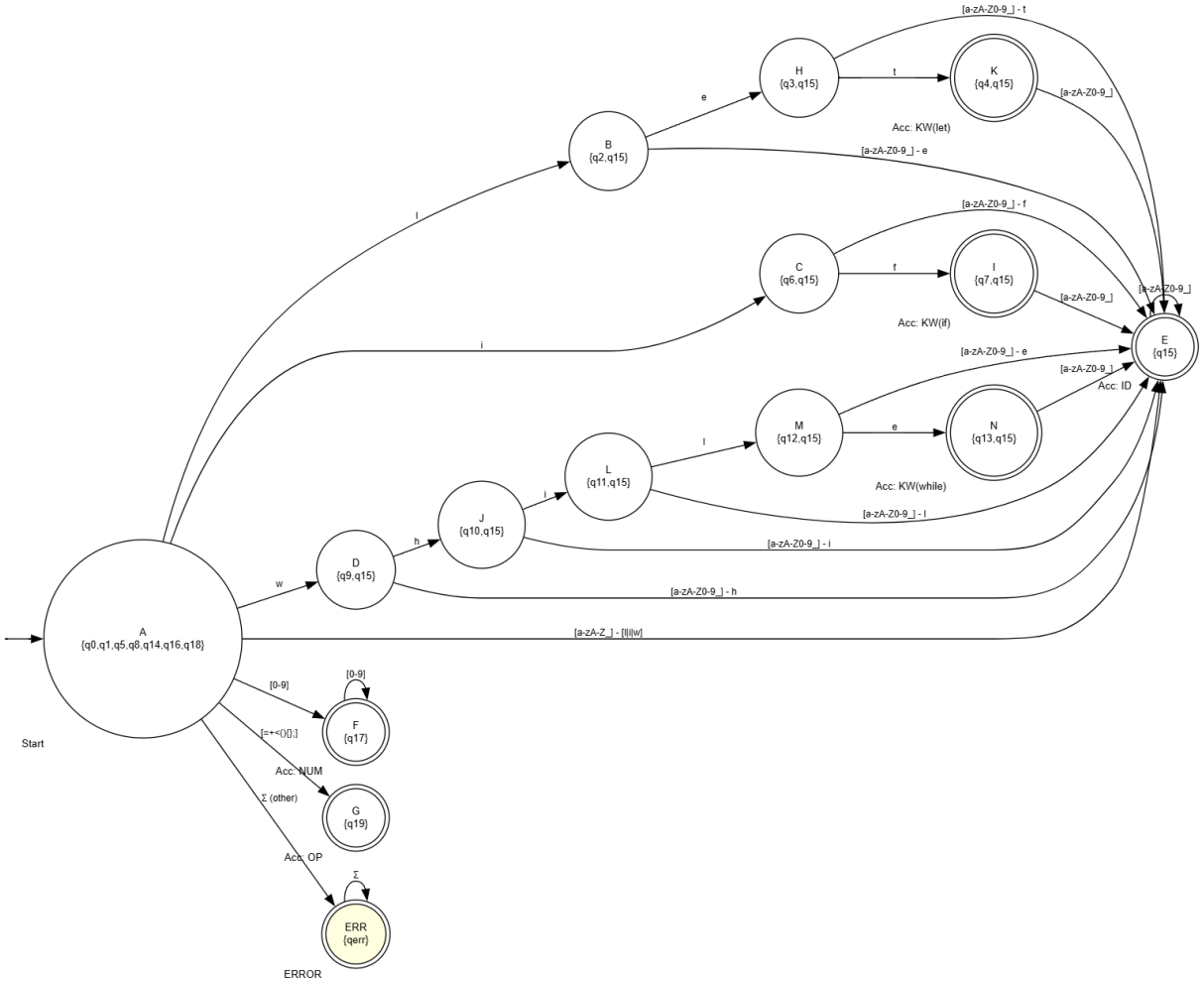

Figure 2: Minimised DFA from Subset Construction. Double circles are accepting states; labels show the emitted token type. All transition labels use explicit character-class notation. The yellow ERR state is the catch-all error sink.

DFA State Transition Table

Column key: *l, i, w, e, f, t, h* — literal characters consumed along keyword paths. *[a-zA-Z_]** — any letter/_ not already named in a preceding column. *[0-9]* — any decimal digit. *[=+¡(){};]* — any op/punct. Σ — any other character (always goes to ERR catch-all).

State	**NFA Subset**	**l**	**i**	**w**	**e**	**f**	**t**	**h**	**[a-zA-Z_]***	**[0-9]**	**Op**	**Σ**
A (Start)	$\{q_0, q_1, q_5, q_8, q_{14}, q_{16}, q_{18}\}$	B	C	D	E	E	E	E	E	F	G	ERR
B	$\{q_2, q_{15}\}$	E	E	E	H	E	E	E	E	E	ERR	ERR
C	$\{q_6, q_{15}\}$	E	E	E	E	I	E	E	E	E	ERR	ERR
D	$\{q_9, q_{15}\}$	E	E	E	E	E	E	J	E	E	ERR	ERR
H	$\{q_3, q_{15}\}$	E	E	E	E	E	K	E	E	E	ERR	ERR
J	$\{q_{10}, q_{15}\}$	E	L	E	E	E	E	E	E	E	ERR	ERR
L	$\{q_{11}, q_{15}\}$	M	E	E	E	E	E	E	E	E	ERR	ERR
M	$\{q_{12}, q_{15}\}$	E	E	E	N	E	E	E	E	E	ERR	ERR
K Acc KW:`let`	$\{q_4, q_{15}\}$	E	E	E	E	E	E	E	E	E	ERR	ERR
I Acc KW:`if`	$\{q_7, q_{15}\}$	E	E	E	E	E	E	E	E	E	ERR	ERR
N Acc KW:`while`	$\{q_{13}, q_{15}\}$	E	E	E	E	E	E	E	E	E	ERR	ERR
E Acc: Identifier	$\{q_{15}\}$	E	E	E	E	E	E	E	E	E	ERR	ERR
F Acc: Number	$\{q_{17}\}$	ERR	ERR	ERR	ERR	ERR	ERR	ERR	ERR	F	ERR	ERR
G Acc: Op/Punct	$\{q_{19}\}$	ERR	ERR	ERR	ERR	ERR	ERR	ERR	ERR	ERR	ERR	ERR
ERR Error sink	$\{q_{\text{err}}\}$	ERR on every input — scanner reports lexical error, does not crash										

*[a-zA-Z_]** covers letters/underscore not named in a preceding column for that row. ERR = catch-all error sink.

Note on the `while` path. $A \xrightarrow{w} D \xrightarrow{h} J \xrightarrow{i} L \xrightarrow{l} M \xrightarrow{e} N$ (Acc: KW(`while`)). At every intermediate state, any `[a-zA-Z0-9_]` character that does not continue the keyword path transitions to E (IDENTIFIER).